CI/CD for Node.js Retail Application

Course Name: DevOps

Institution Name: Medi-caps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	Arsh Patidar	EN22CS301204
02	Anuj Singh Rathore	EN22CS301166
03	Anurag Didolkar	EN22CS301169
04	Avani Gupta	EN22CS301236
05	Amit Patidar	EN22CS301114
06	Aniket Kushwah	EN22CS301124

Group Name: 09D2

Project Number: DO-21

Industry Mentor Name: Mr. Vaibhav

University Mentor Name: Prof. Akshay Saxena

Academic Year: 2025-26

Problem Statement & Objectives

1. Problem Statement

Modern web applications face multiple deployment challenges such as:

- Inconsistent environments between development and production
- Manual deployment errors
- Dependency conflicts
- Slow and unreliable release cycles

These problems reduce system efficiency and increase failure risks.

Therefore, there is a need for an automated, scalable, and reliable system using DevOps practices.

2. Project Objectives

The main objectives of this project are:

- To automate build, test, and deployment using CI/CD pipeline
- To containerize the Node.js application using Docker
- To ensure consistency across all environments
- To reduce manual effort and improve reliability

3. Scope of the Project

The project focuses on:

- Developing a Node.js retail application
- Implementing CI/CD pipeline using GitHub Actions
- Containerizing application using Docker
- Ensuring automated and scalable deployment

Limitations

- **Complex Setup**
  Initial setup of CI/CD, Docker is difficult for beginners.

- **Debugging Issues**
  Errors in pipeline or containers are hard to identify and fix.
- **Security Risks**
  Improper configuration may expose sensitive data.
- **Dependency on Tools**
  System depends on GitHub, Docker Hub services.

4. Proposed Solution

The proposed system is a **complete DevOps-based CI/CD pipeline**.

Solution includes:

- GitHub for version control
- CI/CD pipeline for automation
- Docker for containerization

Workflow:

1. Developer pushes code to GitHub
2. CI/CD pipeline triggers automatically
3. Code is built and tested
4. Docker image is created
5. Image is pushed to registry

A. Key features

The system provides several important features:

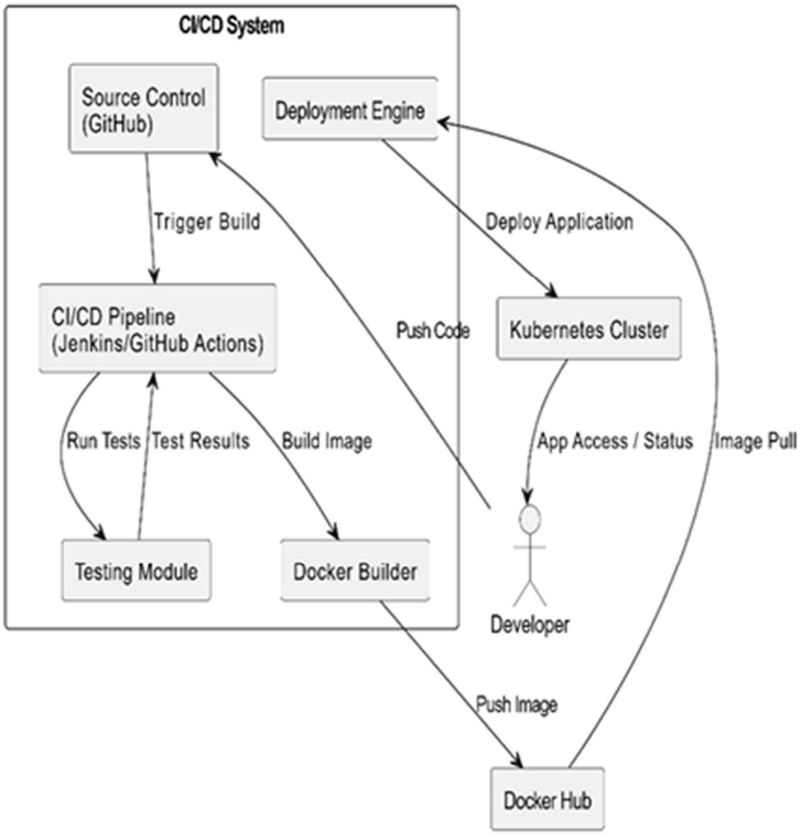
- Automated CI/CD pipeline for continuous integration and deployment
- Docker-based containerization for environment consistency
- Faster and error-free deployment process
- Improved collaboration using version control systems
- High availability and fault tolerance

B. Overall Architecture / Workflow

The system follows a DevOps-based architecture that automates the complete process from code development to deployment. It integrates tools like GitHub, CI/CD pipeline, Docker to ensure smooth and reliable application delivery.

Workflow:

- **Code Development & Version Control**
  The developer writes the Node.js application code and pushes it to GitHub, where all changes are managed.
- **CI/CD Pipeline Execution**
  After code push, the CI/CD pipeline is automatically triggered. It installs dependencies, builds the application, and runs tests.
- **Docker Containerization**
  Once the build is successful, a Docker image is created to package the application with its dependencies.
- **Application Access**
  Finally, the deployed application is made available to users through a web interface.



C. Tools & Technologies Used

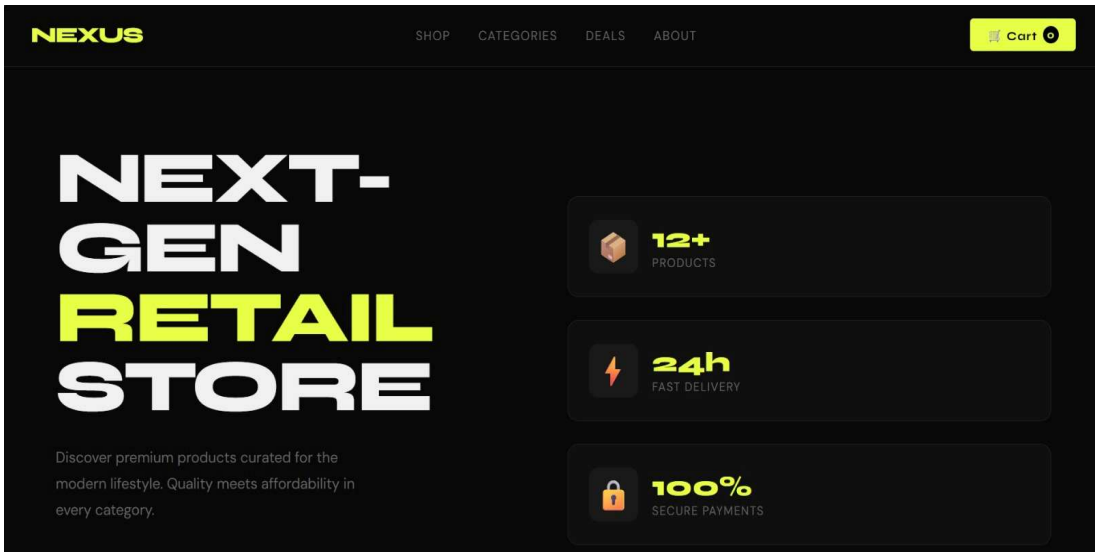
- Node.js – Backend development and server-side logic
- Git & GitHub – Version control and code management
- GitHub Actions – CI/CD automation
- Docker – Containerization of application
- HTML & CSS – Frontend user interface

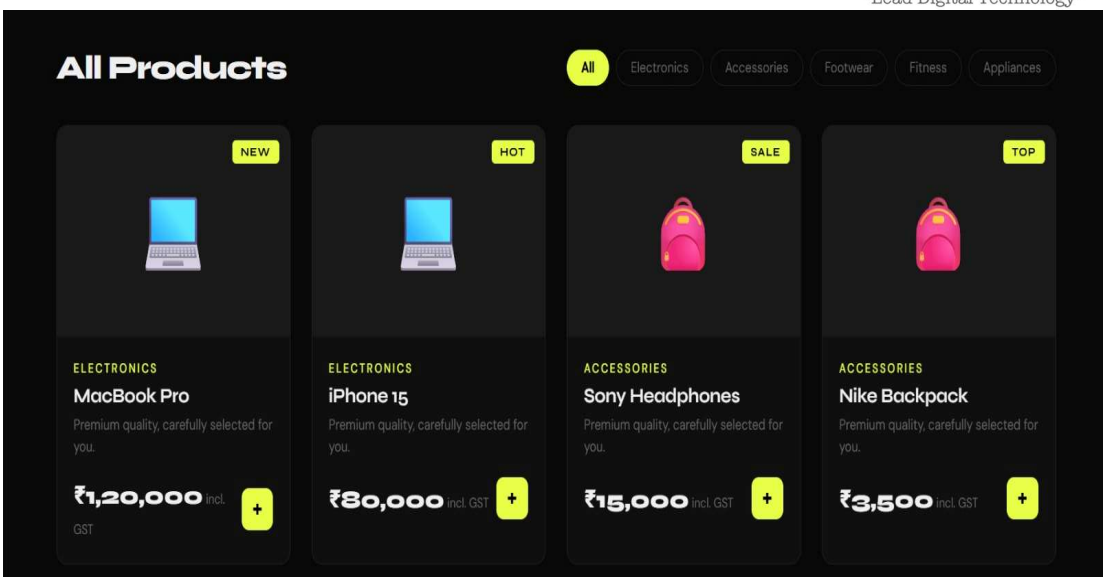
Results & Output

1. Screenshots / outputs

The project successfully demonstrates:

- Faster and automated deployment process
- Consistent environment across all stages
- Reduced manual configuration errors
- Improved scalability and system reliability






```yaml
deploy:
  name: Deploy to AWS EC2
  runs-on: ubuntu-latest
  needs: build-and-push
  if: github.ref == 'refs/heads/main'
  steps:
    - name: SSH and Deploy
      uses: appleboy/ssh-action@v1.0.3
      with:
        host: ${{ secrets.SERVER_HOST }}
        username: ${{ secrets.SERVER_USER }}
        key: ${{ secrets.SERVER_SSH_KEY }}
        key_path: ""
        port: 22
        script: |
          docker pull ${{ secrets.DOCKER_USERNAME }}/retail-app:latest
          docker stop retail-app || true
          docker rm retail-app || true
          docker run -d \
            --name retail-app \
            --restart unless-stopped \
            -p 8080:3000 \
            ${{ secrets.DOCKER_USERNAME }}/retail-app:latest
```

```yaml
name: CI/CD Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:

  test:
    name: Run Unit Tests
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
      - name: Install dependencies
        run: npm ci
      - name: Run tests
        run: npm test

  build-and-push:
    name: Build & Push Image
    runs-on: ubuntu-latest
```

2. Key outcomes

The project demonstrates an efficient and reliable deployment system with improved performance, scalability, and reduced deployment time.

Conclusion

The project successfully implemented a CI/CD pipeline for a Node.js retail application, enabling automated build, testing, and deployment. The use of Docker ensured consistency, scalability, and efficient resource management. Overall, the system improved deployment speed, reduced manual errors, and enhanced application reliability.

- Automated complete software deployment process
- Reduced manual effort and human errors
- Faster and reliable application delivery
- Ensured consistency across environments
- Achieved scalability using Kubernetes

- Improved overall system performance

Future Scope & Enhancements

The project can be further enhanced by integrating advanced tools and technologies to improve scalability, security, and monitoring. Future improvements can make the system more intelligent, efficient, and production-ready.

- Implement advanced monitoring tools (Prometheus, Grafana)
- Add automated security scanning in CI/CD pipeline
- Integrate AI/ML for predictive failure detection
- Improve logging and alerting mechanisms
- Use multi-cloud deployment for higher availability
- Enhance auto-scaling and load balancing features
- Implement blue-green or canary deployments